

Indian Institute of Technology Delhi

COL764: Information Retrieval and Web Search

Assignment 3: Rank Fusion and Learning to Rank

Submitted by

Aditya Anand
2023CS50284

Instructor

Prof. Srikanta Bedathur

October 24, 2026

Overview

The submission has four parts (Figure 1). Fusion and the two neural models are my own code; LambdaMART uses LightGBM's `lambdarank`. All numbers are nDCG@10.

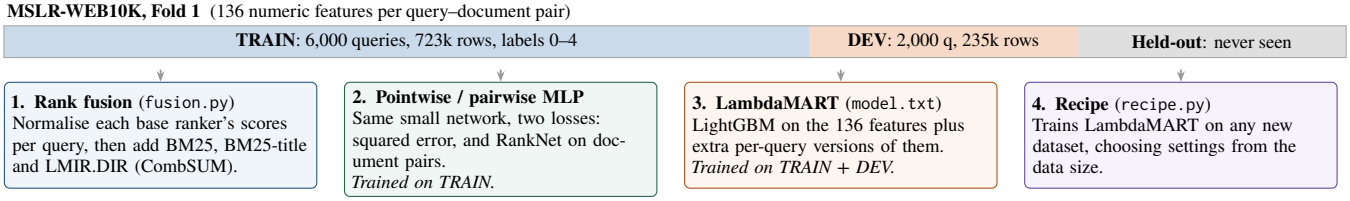


Figure 1: The four parts of the submission.

1 Rank fusion

Each candidate has four base ranker scores: BM25, LMIR.DIR and LMIR.JM on the whole document and BM25 on the title. Their scales differ (BM25 is positive, LMIR is a negative log-probability) so CombSUM and CombMNZ first **min–max normalise each channel inside the query** to $[0, 1]$ (written $\tilde{s}$). A channel where all documents score the same becomes all zeros. RRF uses 1-based ranks; ties keep the candidate order.

Method	TRAIN	DEV	DEV MAP@10
BM25 whole document alone	0.2835	0.2885	0.1205
RRF, $c = 60$	0.2926	0.2995	0.1246
RRF, $c = 150$ (best c)	0.2937	0.3002	0.1250
CombMNZ, 4 channels	0.2958	0.3061	0.1286
CombSUM, 4 channels	0.2961	0.3056	0.1285
CombSUM, 3 channels (shipped)	**0.3000**	**0.3105**	**0.1302**

Table 1: Fusion methods (nDCG@10). DEV was only printed, never used to choose.

$$\mathrm{RRF}(d) = \sum_s \frac{1}{c + \mathrm{rank}_s(d)}, \qquad \mathrm{CombSUM}(d) = \sum_s \tilde{s}(d)$$

$$\mathrm{CombMNZ}(d) = \mathrm{CombSUM}(d) \cdot \left|\{s : \tilde{s}(d) > 0\}\right|$$

All choices were made on the 6000 TRAIN queries: RRF with c from 1 to 500, CombSUM, CombMNZ, a z-score sum, and channel weights in $\{0, 0.25, 0.5, 1, 2\}$.

- **RRF is the weakest fusion.** It improves as c grows up to about 150 (0.2843 at $c = 1$), then stays flat. It keeps only the order of each list and throws away how far apart the scores are.
- **CombMNZ ≈ CombSUM.** After min–max, only the single lowest document of each channel gets 0, so the MNZ multiplier is almost always 4 and changes nothing.
- **LMIR.JM is dropped.** The weight search gave it weight 0. It is the weakest single channel (0.2621) and mostly repeats LMIR.DIR. To check that picking from about 400 options did not overfit, I chose on 4 TRAIN folds and scored the 5th: 0.2989 ± 0.0041, close to 0.3000.

2 Pointwise, Pairwise and LambdaMART

	Pointwise	**Pairwise (RankNet)**	**LambdaMART**
One example	one document	two documents of the *same* query, different labels	a whole query
Loss	$(s - y)^2$, label $y \in \{0, \ldots, 4\}$	$\log\left(1 + e^{-(s_i - s_j)}\right)$, i more relevant; weighted by $\lvert\Delta\mathrm{nDCG}_{ij}\rvert$ in training	RankNet gradient $\times \lvert\Delta\mathrm{nDCG}_{ij}\rvert$
Model	MLP 136→64→1	MLP 136→256→256→256→1	shipped: 2,835 trees, 127 leaves

Pairwise training. Pairs come from one query only and ties are dropped. Each batch holds 64 queries with up to 4,000 pairs each, and each pair is weighted by how much nDCG@10 would change if the two documents swapped. The graded `ranknet_loss` itself stays plain and unweighted. Both networks use Adam and early-stop on a fixed tenth of TRAIN; DEV is scored once, at the end.

LambdaMART uses the same idea with trees: a swap near the top of the ranking gets a large push, a swap far down almost none. Each new tree is fit to these pushes.

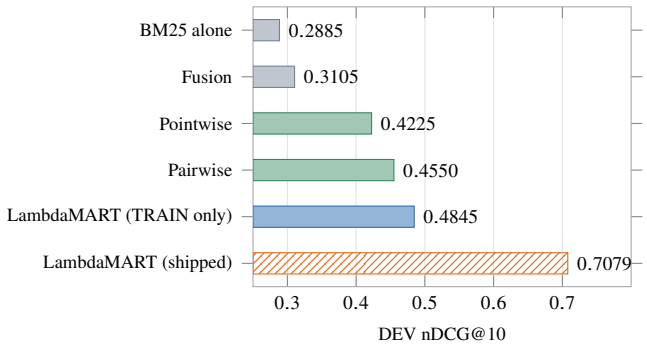


Figure 2: DEV nDCG@10. The hatched bar is *not* a fair score: DEV was inside its training data.

Comparison. LambdaMART is best, then pairwise, then pointwise. Pairwise beats pointwise by 0.033: pointwise learns each document's grade on its own, while pairwise learns the order inside a query, which is what nDCG measures. For a fair LambdaMART score I use a model fit on TRAIN only with the CV-chosen setting (63 leaves, 2,000 trees, Section 3): 0.4845. The shipped file's 0.7079 is an in-training score, so it is not compared.

3 Cross-validation and the generalisation gap

I cannot see the held-out queries, so I measure the gap with **grouped 5-fold cross-validation inside TRAIN**: all documents of a query go to the same fold, each run trains on 4,800 queries and validates on 1,200. Each model is scored at fixed tree counts,

so **nothing is early-stopped on the fold I report**. I report the mean ± std over the five validation folds and the val/train ratio.

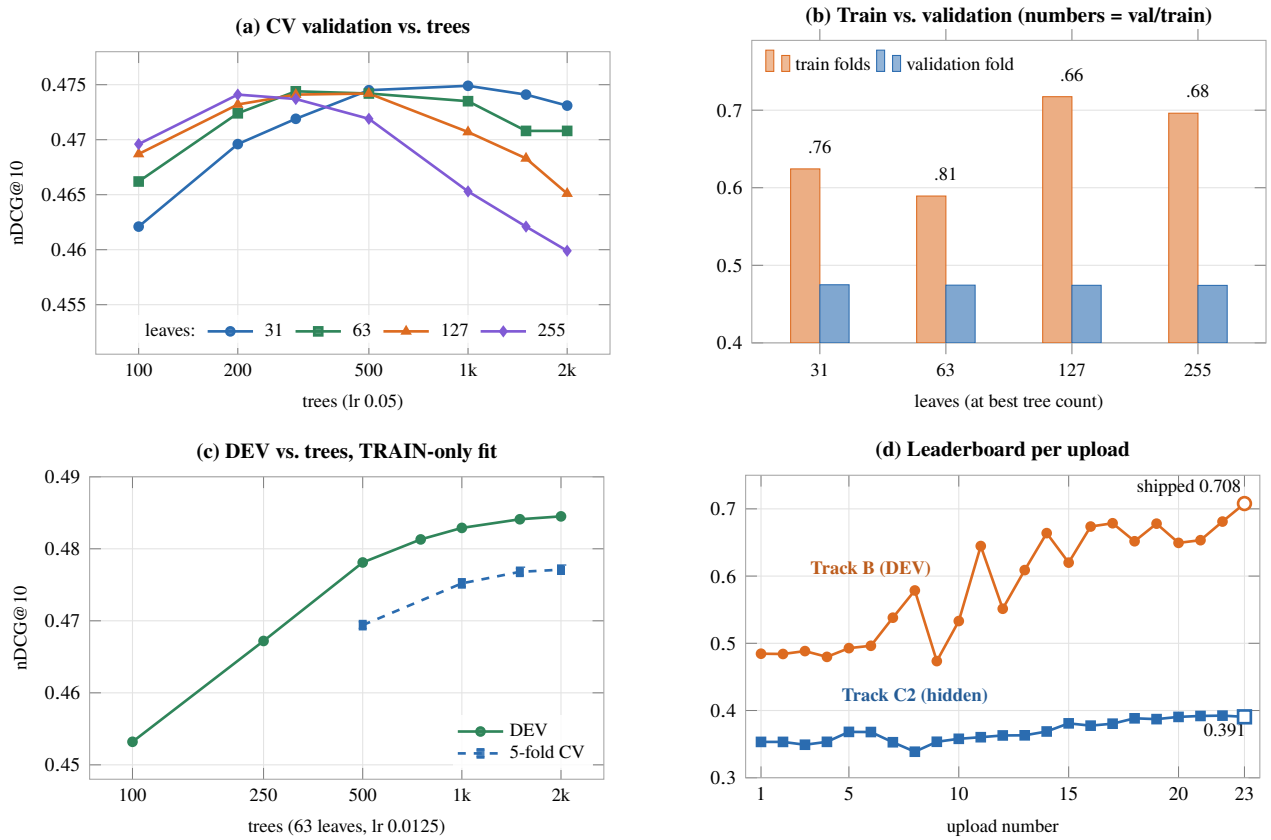


Figure 3: Orange = data the model was fit on, blue = data it never saw. (a)–(b) Grouped 5-fold CV on TRAIN. (c) A TRAIN-only LambdaMART scored on DEV, next to its CV estimate. (d) Leaderboard scores of my 23 distinct LambdaMART uploads in time order (23 = shipped file).

lr / leaves / min leaf / cols	trees	val nDCG@10	train	ratio
0.05 / 31 / 100 / 0.8	1000	0.4749 ± .0059	0.624	0.76
0.05 / 63 / 100 / 0.8	300	0.4744 ± .0063	0.589	**0.81**
0.05 / 127 / 100 / 0.8	500	0.4742 ± .0077	0.717	0.66
0.05 / 255 / 100 / 0.8	200	0.4741 ± .0059	0.696	0.68
0.05 / 255 / 100 / 0.8	2000	0.4599	0.944	0.49
0.05 / 63 / 500 / 0.5	500	0.4741 ± .0068	0.623	0.76
0.025 / 63 / 500 / 0.5	1000	0.4766 ± .0070	0.625	0.76
0.0125 / 63 / 500 / 0.5	**2000**	**0.4771 ± .0068**	0.626	0.76

Table 2: Grouped 5-fold CV on TRAIN, at each setting's best tree count. Orange row: the 255-leaf model left to 2,000 trees.

What the gap shows.

- **Validation is flat; training is not.** Every tree shape peaks at 0.474–0.477. Bigger trees only raise the training score: 255 leaves reach 0.944 on training folds while validation *falls* to 0.460.
- **DEV agrees with CV** for a model that never saw DEV: 0.4845 vs. 0.4771, with the same curve shape (Fig. 3c).
- **Once DEV is in training, its score means little.** Over my uploads Track B rose by 0.22 (0.4845 to 0.7079), but the hidden C2 score only from 0.353 to 0.391 (Fig. 3d). My estimate for new clean queries stays ≈**0.477**.

4 Regularisation choices and the shipped LambdaMART

Since more capacity does not help unseen queries, I spent it on **robustness**: Track C2 scores the model on five changed copies of the held-out data. Adding small noise to DEV drops the plain TRAIN-only model from 0.4845 to 0.3450, close to its real C2 score (0.3533). My responses:

- **Usual tree regularisation:** learning rate falling 0.05→0.015, column sample 0.5, row bagging 0.9, L2 0.1, at least 20–40 rows per leaf, tree count fixed in advance.
- **Scale-free columns:** each feature also gets its percentile and z-score *inside the query* (408 columns). These do not change if a feature is rescaled.
- **Corrupted training copies:** noise, log view, zeroed or missing cells, shuffled fields, per-query rescaling; labels stay the same.

During the staged run below, the real C2 score rose from 0.381 to 0.392 (uploads 15–22).

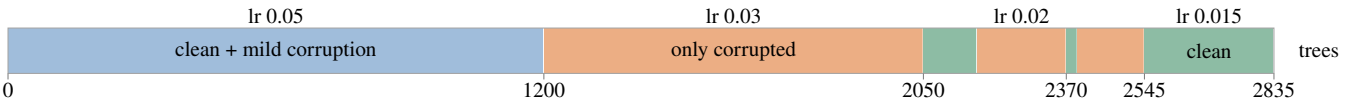
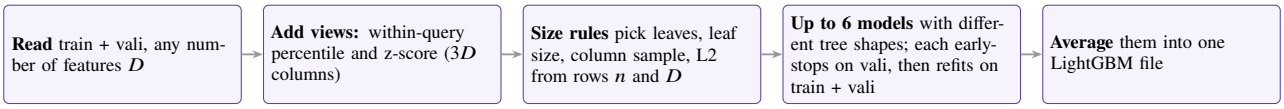


Figure 4: How the 2,835 shipped trees were grown, all on TRAIN + DEV. Orange stages use only corrupted copies, green stages only clean rows; the last 250 trees are clean.

5 Training recipe for the hidden datasets (Track C1)



An MSLR model cannot be used on the hidden datasets (different, unnamed features), so the recipe must carry *how* to train. Settings come from the data size: small data gets small, strongly regularised trees (e.g. leaves = $\mathrm{clip}(2^{\mathrm{round}(\log_2(\sqrt{n}/16))}-1,\ 7,\ 63)$), and nothing assumes 136 features. A time budget of 15 minutes (limit: 20) decides how many models are trained. On 5 benchmark-like tasks cut from TRAIN, an earlier single-model version averaged 0.5011 vs. 0.4882 for the starter recipe. A synthetic file at the stated limit (500k rows, 300 features) took 353 s.

6 Final configuration and what I traded off

Fusion is CombSUM of three min–max channels. The **pointwise** and **pairwise** networks use only TRAIN. The **LambdaMART** file has 2,835 trees with 127 leaves on 408 columns, fit on TRAIN + DEV with corrupted copies. I gave up two things. First, *a fair DEV score*: training on DEV lifts Track B to 0.7079, but my estimate for unseen clean queries is the CV value of about 0.477. Second, *some clean accuracy*: the corrupted copies aim at noisy, missing or rescaled features, which is what Track C2 measures, rather than at clean DEV. The 63-leaf clean model has the best val/train ratio (0.81) at the same 0.477, but no defence against noisy or rescaled features, so I did not ship it.

Model	Trained on	Tuned on	Early stopping
LambdaMART `model.txt`	TRAIN + DEV	5-fold CV on TRAIN; tree count and copies on TRAIN slices and DEV	none
`pointwise_model.pt`	9/10 of TRAIN	not tuned	other 1/10 of TRAIN
`pairwise_model.pt`	9/10 of TRAIN	pair sampling and learning rate on a TRAIN slice	other 1/10 of TRAIN
`recipe.py`	hidden dataset's train + vali	MSLR TRAIN CV and slices; synthetic transfer sets	its vali, then refit

7 AI use and code provenance

AI use. I used Claude Code (Anthropic, Claude Opus 5.5) and Cursor (Grok) to read the starter code, write the submission, run the experiments and draft this report. I chose the fusion channels, the LambdaMART settings and the recipe rules, and I checked the numbers. I am responsible for the submission.

Code provenance. No external code was copied. RRF follows Cormack, Clarke and Buettcher (SIGIR 2009); CombSUM and CombMNZ follow Fox and Shaw (TREC-2, 1994); RankNet follows Burges et al. (ICML 2005); the |ΔnDCG| weights follow Burges (MSR-TR-2010-82) and are used only outside the graded `ranknet_loss`. LambdaMART is LightGBM's `lambdarank`. Libraries: NumPy, LightGBM, PyTorch.